STA2005S Regression Practical Two - RMarkdown and Inference

Questions

Introduction to Markdown, RMarkdown/Quarto & Hypothesis Testing

This is a subheading

This is italic

- bullet point 1 [NB leave a new line after the words above]
- bullet point 2

1. number 1
2. number 2

The following is `inline code`

```
Code on its own line
```

Running R-code: `2+3`

```
2+3
```

```
[1] 5
```

In line math mode $y_i = \beta_0 + x_{1_i}\beta_1 + \epsilon_i$

$$Math\ mode\ on\ its\ own\ line$$

Primary objective: Conduct hypothesis testing using a linear regression model.

Secondary objective: Familiarize yourself with Markdown syntax, and learn how to integrate R code within a Markdown document using Quarto.

We aim to make it as easy as possible for you to create a working Quarto document that RStudio turns into an HTML with all your analysis. As such, in this prac we describe Markdown and Quarto at a minimal level so as you can actually use them, and give you a plain text file (`PracReg2RdInfStarter.qmd`) that you gradually convert (partly by adding in code we give you) into a working Quarto document.

You do not actually need to learn Markdown and Quarto to do this assignment, or any other coding in this course - you can simply work off an R script. Markdown and Quarto are secondary objectives. But learning Markdown and Quarto will likely be very useful both in future assignments and in your day-to-day career as a data analyst. Combining code and text describing the results is very convenient, and the ability to write mathematics beautifully (especially if you are creating a PDF) is important in academic work.

If you do not wish to learn Markdown and Quarto now, skip to `Part 3: Hypothesis testing` towards the end of this document.

It is also not a bad option to complete the practical in a standard R script, but attempt with the time you have to cover as much of Markdown and Quarto as you can fit this week. Later practicals, and the assignment, will also be opportunities for you to learn.

Part 1: Getting to Know Markdown

Markdown is the mark-up language that formats the text, e.g. for creating headers (such as `# Introduction` becoming a big heading `Introduction`) or bold format (e.g. `**IMPORTANT POINT**` being represented as bold).

Document written in Markdown can be converted into many different document types, e.g. PDF, Word and HTML.

Markdown is very easy to learn, and you can learn it by doing here.

Practice task

You've been provided with a plain-text document, `PracReg2RmdInfPlainText.qmd`. Your first task is to transform this document into a well-structured Markdown document.

Cheat sheet

Here are the Markdown commands you'll need to know:

- Headers: Use `#` for main headers and `##` for subheaders (and `###` for subsubheaders, and so on).
- Bold text: Use `**<text>**`.
- Italic text: Use `*<text>*`.
- Bullet Points: Use `-` followed by a space (and then start a new bullet point on the next line).
- Numbered Lists: Simply use numbers followed by a period and space, e.g. `1. <text>` on one line and `2. <text>` on the next.
- Code Inline: Use backticks around your code.
- Code in its own paragraph (display code): Use triple back-ticks, skip a line, write your code and then close off with triple backticks again.
- Math inline: Use `$` around the text, e.g. `$y_i=\beta_0 + x_{1_i}\beta_1+\epsilon_i$` becomes $y_i = \beta_0 + x_{1_i}\beta_1 + \epsilon_i$.
  - Other points about that equation:
    - `y_i` returns `y` with `i` as the subscript, i.e. y_i.
    - `x_{1_i}` returns `x_1` with `i` as the subscript to `1` (so, as sub-subscript), i.e. x_{1_i}.
    - `\beta` and `\epsilon` are Greek symbols, i.e. β and ϵ. They have a back-slash before them beecause they're actually commands. If you remove the backslashes, you get just the letters back, e.g. `beta` becomes *beta* whereas `\beta` becomes β.
  - Note that the above points about the equation (such as `_` denoting subscript and `\<greek_letter_name>` becoming a Greek letter) only apply within math mode, i.e. for anything in between the `$`.
  - A common source of errors is forgetting the second dollar sign.
- Math in its own paragraph (display math): Use double dollar signs (i.e. `$$`) to start, skip a line, write the math, and then close off with `$$` again.
  - A common error is using a single `$` on one side of the equation and `$$` on the other (or vice versa).

For more information (over and above what you'll need to complete this tutorial), see Quarto's Markdown Basics document. It also gives more visual representations of what the Markdown translates into, i.e. how the final product looks.

Part 2: Introducing Quarto

Quarto is a powerful tools that allows you to blend the simplicity of Markdown with the analytical capabilities of R. Essentially, it is Markdown + code. You can give background,

describe the results and provide discusion using Markdown, and perform the actual analysis using R - all in the same plain text file. The code is run, and a document of your chosen type (e.g. HTML, PDF, Word) will be created.

Relationship to `RMarkdown`

Quarto is the successor to RMarkdown (which you may have heard of), and as such we focus on it here. Only minor adjustments are needed to transform a Quarto document to an RMarkdown document, but it will be confusing to do both and Quarto is what is being actively developed. If you are comfortable with RMarkdown and don't want to learn Quarto, that's completely fine.

Setting up Quarto

Note that to get `Quarto` running, you need not only to install the `R` package `quarto`, but also to install the `Quarto` program itself. Go here for the precise walkthrough: https://quarto.org/docs/get-started/.

Quarto in RStudio

See here for exactly which buttons to click to render Quarto documents in RStudio. Note that by "render", I mean convert from plain text (i.e. your source code, what you wrote) into the desired document (e.g. PDF, Word, HTML).

Additional syntax to run code

There are two main additions to plain Markdown:

- YAML frontmatter
- Code chunks

YAML Frontmatter

The YAML frontmatter is metadata at the beginning of your Quarto document. It is used to set various options for rendering the document. For example, `title` sets the title of the document and `format` determines the format of the output (e.g., HTML, PDF, Word).

Copy and paste the following YAML frontmatter at the very beginning of your `qmd` document (`PracReg2RmdInfStarter.qmd`):

```
---
title: "STA2005S Regression Practical Two - RMarkdown and Inference"
format: html
---
```

Note that you need to include the triple dash, `---`, at the start and at the end. Also note that this needs to go at the very top, i.e. there must be no code or blank lines above the first `---`.

With this setting, your Quarto document will be rendered as an HTML page when you knit it. If you instead, for example, set `format` to `pdf` you would create a PDF document. If you set `format` to `docx`, you would create a a Word document.

For more details on YAML options for Quarto, refer to Quarto's frontmatter documentation.

Code Chunks

Syntax

To insert R code into your document, use code chunks. A code chunk begins with three bacticks followed by `{r}`, skips a line to start code and ends with three backticks again (on a new line). The `{r}` ensures that the code is run as `R` code (and not, for example, python).

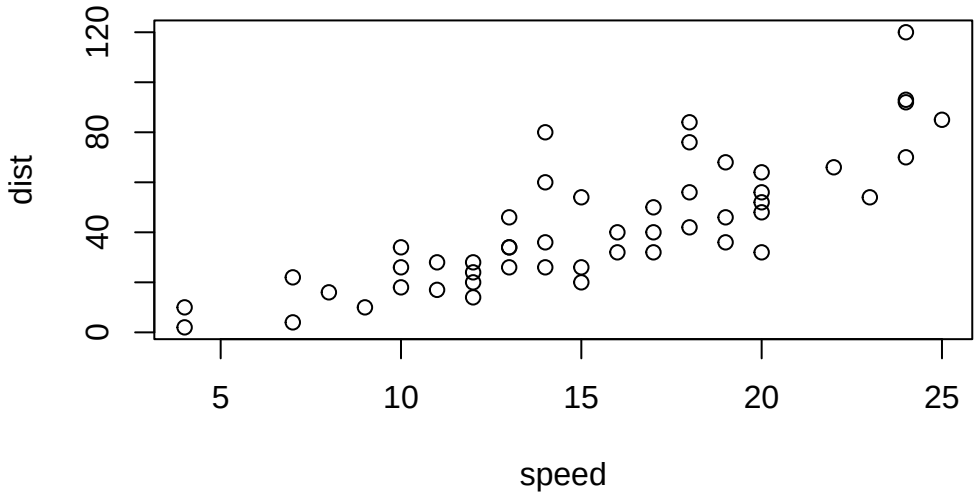
For example, we can display data:

````
```{r}
data("cars")
head(cars)
```
````

```
  speed dist
1     4    2
2     4   10
3     7    4
4     7   22
5     8   16
6     9   10
```

We can display a plot:

````
```{r}
plot(cars)
```
````



Chunk options

This will embed the plot directly in your output document. Code chunks are useful for both displaying code and showing the output of the analysis directly in the document.

In Quarto, the syntax for code chunks is to place `#| <option_name>: <option_value>` on new lines immediately following the initial three back ticks to start the chunk.

For example, we can create a better looking table than that produced above by using R code that turns the dataframe into LaTeXformat. To do that, we use the option `results` and give it the value `asis`:

```
```{r}
#| results: asis
xtable::xtable(head(cars))
```
```

% latex table generated in R 4.4.1 by xtable 1.8-4 package % Sun Sep 29 18:15:19 2024

	speed	dist
1	4.00	2.00
2	4.00	10.00
3	7.00	4.00
4	7.00	22.00
5	8.00	16.00
6	9.00	10.00

If you did not do that (use `results: asis`), the LaTeXcode would not processed and you will get the raw LaTeXback:

```
```{r}
xtable::xtable(head(cars))
```
```

```
% latex table generated in R 4.4.1 by xtable 1.8-4 package
% Sun Sep 29 18:15:19 2024
\begin{table}[ht]
\centering
\begin{tabular}{rrr}
  \hline
 & speed & dist \\
  \hline
1 & 4.00 & 2.00 \\
  2 & 4.00 & 10.00 \\
  3 & 7.00 & 4.00 \\
  4 & 7.00 & 22.00 \\
  5 & 8.00 & 16.00 \\
  6 & 9.00 & 10.00 \\
   \hline
\end{tabular}
\end{table}
```

Quick note - you cannot use LaTeXin HTML documents, so the above code will cause an error there. I just wanted to create an example where you might want to use `results: asis`.

Other useful options are:

- `echo`: Set `false` to not display the code, and `true` to display the code.
- `include`: Set `false` to run the code, but suppress all output, including all warnings and messages. Useful if you run code that tends to generate a lot of warnings and messages you're not interested in (such as attaching many packages using `library`).
- `eval`: Set `false` to not run a code chunk. Sometimes you want code not to run but don't want to delete it (a bug you'll get to later, perhaps), or else it's demonstration code that you merely want to display.

There are many more! See here for more options.

In the hypothesis testing section, we will give you code that you can copy into `PracReg2RmdInfStarter.qmd` to get started on answering the assignment in Quarto.

Debugging advice

ChatGPT is especially useful when you are new to a language. A quick and easy way to debug your code is to copy it, along with your error messages, into ChatGPT. It won't necessarily be accurate or even helpful, but it's easy to do and often will point you in the right direction.

Part 3: Hypothesis testing

Introduction

Suppose we are analysing the advertising data again, using which we wish to assess how sales depend on TV, radio and newspaper advertising spend. We posit the model

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon},$$

where $\mathbf{e} \sim \mathrm{N}(\mathbf{0}, \sigma^2\mathbf{I})$ and $\boldsymbol{\beta}' = \begin{bmatrix}\beta_0 & \beta_{\mathrm{TV}} & \beta_{\mathrm{Radio}} & \beta_{\mathrm{Newspaper}}\end{bmatrix}$.

Questions

As part of hypothesis testing, we will conduct three tests:

1. Check whether we detect an effect of radio spend on advertising sales ($H_0 : \beta_{\mathrm{Radio}} = 0$).
2. Check whether we detect of the sum of TV and newspaper spend on sales ($H_0 : \beta_{\mathrm{TV}} + \beta_{\mathrm{Newspaper}} = 0$).
3. Check whether we detect an effect of any of TV, radio and newspaper spend on sales ($H_0 : \beta_{\mathrm{TV}} = 0, \beta_{\mathrm{Radio}} = 0, \beta_{\mathrm{Newspaper}} = 0$)

We will answer the first question in this practical, and you will do the next two.

Quarto note

For those using Quarto, the code is printed inside the fences (i.e. the triple backtick pairs), so that you can copy it directly into your Quarto document. For those not using Quarto - do not copy the triple backtick lines.

If you are using `Quarto`, I suggest carrying on working in that `PracReg2RmdInfStarter.qmd` file. But up to you.

I have also uploaded the file that generated this questions sheet - `PracReg2RmdInfQuestions.qmd` - which may be useful for copying mathematical notation across. The only difference you may find to what is usual is that I use `\symbf` instead of the more usual `\mathbf` and `\bm` for creating bold math notation - it just seems to work better in my setup. You can use `Find and Replace` to replace all instances of `\symbf` with `\mathbf`, if you like.

Answers

First we read in and load our data:

````
```{r}
if (!requireNamespace("DataTidy23RodoSTA2005S", quietly = TRUE)) {
  if (!requireNamespace("remotes", quietly = TRUE)) {
    utils::install.packages("remotes")
  }
  remotes::install_github("MiguelRodo/DataTidy23RodoSTA2005S")
}
library(tibble)
data("data_tidy_ad", package = "DataTidy23RodoSTA2005S")
```
````

We print the first six rows:

````
```{r}
data_tidy_ad |>
  head() |>
  knitr::kable()
```
````

TV	radio	newspaper	sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	9.3
151.5	41.3	58.5	18.5
180.8	10.8	58.4	12.9
8.7	48.9	75.0	7.2

Hypothesis I: Effect of radio advertising on sales

We wish to test for the effect of radio advertising on sales. Specifically, we wish to test the null hypothesis

$$H_0 : \beta_{\text{Radio}} = 0$$

against the alternate hypothesis

$$H_0 : \beta_{\text{Radio}} \neq 0.$$

We will use the t statistic, which is

$$\frac{\hat{\beta}_{\text{Radio}} - \beta_{\text{Radio}}}{\sqrt{s^2(\mathbf{X}'\mathbf{X})^{-1}_{33}}} \sim t_{n-p},$$

for $s^2 = \frac{(\mathbf{Y}-\mathbf{X}\boldsymbol{\beta})'(\mathbf{Y}-\mathbf{X}\boldsymbol{\beta})}{n-p}$ the unbiased estimator for σ^2.

We have that $n = 200$ and $p = 4$. In addition, under H_0, $\beta_{\text{Radio}} = 0$, which means that we will compare $\frac{|\hat{\beta}_{\text{Radio}}|}{\sqrt{s^2(\mathbf{X}'\mathbf{X})^{-1}_{33}}}$ against $t^{0.975}_{196} \approx 1.97$ (using a significance threshold of 0.05). Should the test statistic exceed the specific quantile of the t-distribution, we will reject the null hypthesis that $\beta_{\text{Radio}} = 0$.

First, we create our vector (single-column matrix) of response values and our design matrix of explanatory variable values:

```
```{r}
Y_vec <- matrix(data_tidy_ad[["sales"]], ncol = 1)
X_mat <- data_tidy_ad |>
  dplyr::select(-sales) |>
  dplyr::mutate(intercept = 1) |>
```
```

```
  dplyr::select(intercept, everything()) |>
  as.matrix()
```

Secondly, we estimate β_{Radio}:

```{r}
beta_vec_hat <- solve(t(X_mat) %*% X_mat) %*% t(X_mat) %*% Y_vec
beta_vec_hat |> signif(2)
```

```
            [,1]
intercept  2.900
TV         0.046
radio      0.190
newspaper -0.001
```

This yields the following value (rounded) for β_{Radio}:

```{r}
beta_radio_hat <- beta_vec_hat[3]
beta_radio_hat |> signif(2)
```

```
[1] 0.19
```

Thirdly, we calculate s^2 and then the standard error for $\hat{\beta}_{\text{Radio}}$:

```{r}
s2 <- t((Y_vec - X_mat %*% beta_vec_hat)) %*% (Y_vec - X_mat %*% beta_vec_hat) / (196)
sqrt(s2) |> signif(4)
```

```
      [,1]
[1,] 1.686
```

```{r}
beta_radio_hat_se <- sqrt(s2 * solve(t(X_mat) %*% X_mat)[3, 3])
beta_radio_hat_se |> signif(4)
```

```
         [,1]
[1,] 0.008611
```

Then, we calculate the test statistic:

```{r}
test_stat_radio <- beta_radio_hat / beta_radio_hat_se
test_stat_radio |> signif(5)
```

```
       [,1]
[1,] 21.893
```

This is clearly far greater than the threshold for significance, 1.97.

In any case, we calculate the the p-value:

```{r}
pt(abs(test_stat_radio), df = 196, lower.tail = FALSE)
```

```
             [,1]
[1,] 7.526695e-55
```

There is clearly a statistically significant effect.

We can check our results by fitting a linear model using the `lm` function in R:

```{r}
fit <- lm(sales ~ radio + newspaper + TV, data = data_tidy_ad)
summary(fit)
```

```
Call:
lm(formula = sales ~ radio + newspaper + TV, data = data_tidy_ad)

Residuals:
    Min      1Q  Median      3Q     Max
-8.8277 -0.8908  0.2418  1.1893  2.8292
```

```
Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)  2.938889   0.311908   9.422   <2e-16 ***
radio        0.188530   0.008611  21.893   <2e-16 ***
newspaper   -0.001037   0.005871  -0.177     0.86
TV           0.045765   0.001395  32.809   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.686 on 196 degrees of freedom
Multiple R-squared:  0.8972,    Adjusted R-squared:  0.8956
F-statistic: 570.3 on 3 and 196 DF,  p-value: < 2.2e-16
```

The estimate, standard error and test statistic match exactly, and the the p-value is essentially the same. In addition, note that the `Residual standard error` from the `lm` fit is 1.686, which is exactly the square root of our unbiased estimator for the variance.

Your answer to to the third question of this practical should essentially reproduce the final line of the `summary` output above (beginning `F-statistic:`).